The `fdp-demo-hazard` Workflow

A code-level walkthrough of the DIII-D tearing-mode hazard pipeline, with an extended note on what "calibration" means

Code reading of `~/Python/fdp-demo-hazard`

Abstract

This document traces the hazard-modeling framework in `fdp-demo-hazard` starting from the driver script `fdp-run-study-plain.sh`. The pipeline has five stages: (A) two independent magnetics event detectors that emit *event time versus threshold* tables rather than single labels; (B) a joint annotation step that turns the two threshold ladders into one right-censored survival table and scans the two-dimensional threshold plane; (C) a `toksearch` harvest of $\sim$20–27 dimensionless EFIT descriptors; (D) a time-matched merge of EFIT01 and EFIT02 onto a common time base so the two reconstructions can be compared on *identical* rows; and (E) a gradient-boosted-tree fit of a discrete-time hazard function with a custom objective, nested cross-validation, and shot-respecting folds. Section 7 answers the specific question about calibration: the term is used for two unrelated things in this repository, and the important one is a *diagnostic* that establishes the model score f can be read as a physical onset rate via $\Delta h = \ln(1 + e^f)$. Section 8 lists what must change to port the rotating-mode detector to MAST-U.

Contents

1 Entry point: `fdp-run-study-plain.sh`

The script is a make-like driver: every stage is guarded by an `if [ -f ... ]` test, so a rerun in an existing folder resumes rather than recomputes. It takes three arguments,

```
./fdp-run-study-plain.sh <out-folder> [rm-thresh] [lm-thresh]
```

with defaults 5.0 G (rotating mode, RM) and 15.0 G (locked mode, LM). The header comments enumerate the sensitivity study the README asks for: `POINT-A` through `POINT-F` are six (RM, LM) threshold pairs spanning 5.0–6.0 G and 12–17 G.

The script does *not* detect events. It consumes two CSV files that must already exist, produced upstream by `fdp-regen-brm-blm-plain.sh`:

`fdp-regen-brm-plain-155001-198100.csv` and `fdp-regen-blm-ic-155001-198100.csv`

covering DIII-D shots 155001–198100. Three design decisions are visible in the first 120 lines and they set the tone for the whole framework:

1. **Thresholds are pinned to the output folder.** The chosen (RM, LM) pair is written to `<folder>/thresholds.txt` on first run and *checked by string comparison* on every subsequent run; a mismatch is a hard exit. This prevents a folder from accumulating results generated under different labelings.

2. **The source tables are watermarked.** The MD5 of the concatenated RM and LM CSVs is computed and embedded in the threshold-scan pickle filename, `fdp-regen-thresh-scan-plain-<md5>.pickle`. Regenerating the detector output invalidates the cached scan automatically.

3. **Everything downstream is a pure function of the joint annotation file.** The single artifact that connects detection to modeling is `fdp-regen-joint-NEW.csv`.

The stage sequence is:

Step	Program	Artifact
Threshold-plane scan	`eventPathwayAnalysisPlain.py --calc-llikl-surface` (24 procs, `srun`)	`fdp-regen-thresh-scan-plain-<md5>.pickle`
Threshold contour figure	`checkPathwayPickle.py`	`fdp-run-contours-NEW.pdf`
Joint annotation at chosen thresholds	`eventPathwayAnalysisPlain.py --amp-combined-csv`	`fdp-regen-joint-NEW.csv`
Feature harvest, EFIT01	`extractEqDescriptor.py` (`toksearch_submit`)	`fdp-descriptor-NEW-efit01.pickle`
Feature harvest, EFIT02	`extractEqDescriptor.py`	`fdp-descriptor-NEW-efit02.pickle`
Time-matched merge	`processEqDescriptorPair.py --merge`	`fdp-tabular-merged-NEW-efit01-efit02-all.pickle`
Model ensembles (6 ablations × 2 EFITs)	`fdp-run-study-xgb-ensembles.sh` → `sbatch`	`fdp-outer-preds-*-models.pickle`
Held-out predictions + SHAP (3 ablations × 2)	`fdp-run-study-xgb-outerpreds.sh`	`fdp-outer-preds-*.pickle`

The script ends after `sbatch` submission; it does not wait for results. Two further suites (`launch_ml_pooled_experiments`, `launch_ml_elimination_experiments`) are present but switched off by flag.

The six ablations define the feature-set experiment, and this is where the "20 or so features" of the study design actually appear:

Tag	Excluded	Count
`FULL`	—	27
`NOPQ`	`pres0 pres50 dpres50 q33 q67 dq67`	21
`NOP`	`pres0 pres50 dpres50`	24
`NOQ`	`q33 q67 dq67`	24
`NOGL`	`glitch`	26
`NOPQ0`	`pres0 pres50 dpres50 q0 q33 q67 dq67`	20

`NOPQ` (21 features) and `NOPQ0` (20 features) are the profile-free sets: they use only 0-D EFIT scalars and drop everything derived from the `pres` and `qpsi` profile nodes. These are the "∼20 feature" configurations, and they are the only two ablations that appear in *both* the ensemble and the held-out-prediction suites, i.e. they are the intended headline models.

2 Stage A — the two magnetics detectors

Run separately via `./fdp-regen-brm-blm-plain.sh 4 155001 198100`. Both detectors share a common philosophy that is worth stating explicitly because it is the single most transferable idea in the repository:

A detector never emits a label. It emits the first crossing time as a function of a threshold ladder. The threshold becomes a post-processing parameter, and the sensitivity study costs no re-computation over the database.

The RM CSV therefore carries 117 columns named `rm1.0000g` ... `rm30.0000g` (1–30 G, 0.25 G steps), and the LM CSV carries 2×73 columns `in*g`/`ex*g` (4–40 G, 0.5 G steps), one entry per shot per level, `NaN` if never crossed.

2.1 Shared analysis window: flat-top detection

Both detectors call `iptimestamp.iptimestamp` (`iptimestamp.py`) on `ip`. It smooths I_p with a 500 ms boxcar, differentiates, finds the maximum and minimum ramp rates, and back-tracks to where the rate falls below $\alpha = 0.20$ of the extremum, giving four timestamps t_{10}, t_{11} (ramp-up start/end) and t_{20}, t_{21} (ramp-down start/end). The analysis window is $[t_A, t_B] = [t_{11}, t_{20}]$, subject to:

- $t_A > 0.40\,\mathrm{s}$, $t_B > t_A$, $t_B > 1.00\,\mathrm{s}$;
- if `t2ratio` < 1.05 the shot is presumed disrupted, and t_B is pushed to $0.55\,t_{20} + 0.45\,t_{21}$ to avoid discarding the pre-disruption data;
- $|\langle I_p \rangle| > 0.50\,\mathrm{MA}$ over the window (from `flattopness`, which also returns a linear-fit slope and normalized RMSE as flat-top quality metrics).

Shots failing any test get `trange = None` and are dropped. All of these numbers are DIII-D specific.

2.2 RM detector: `brm_events_modespyec.py` + `modespyec.py`

This is the "plain" detector; the older even/odd-parity code (`mpxspec.py`, invoked with `--mpxspec-classic`) is run in parallel only for comparison. The docstring states the simplification: no attempt to separate odd from even toroidal phase, on the argument that a sufficiently high amplitude threshold already excludes sawteeth.

Inputs. Ten midplane $\dot{B}$ probes with the `d` suffix (`mpi66m067d`...`mpi66m340d`, plus `mpi11m322d`), with hard-coded toroidal angles $67.5°, 97.4°, \ldots, 339.7°$. The code asserts a 5 μs sample interval to within 0.1% (`okTs`), i.e. 200 kHz.

Pipeline (per probe pair). `summarize_consecutive_circular_pairs` forms the nine *consecutive circular pairs* $(i, i{+}1 \bmod 9)$ of the outboard probes and calls `wsfft_paired_signal` on each:

- sliding-window FFT, `blocksize`= 1024, `blockstride`= 512, `nfft`= 1024, Hamming window — so a 5.12 ms analysis window every 2.56 ms;
- per-block mean removal, auto-spectra X_{11}, X_{22} and cross-spectrum X_{12};
- boxcar smoothing over `nfsmooth`= 7 frequency bins, then magnitude-squared coherence $C_{12} = |S_{12}|^2/(S_{11}S_{22})$;
- the `modespec` 95% coherence significance level `c95` is reproduced verbatim, $c_{95} = \tanh^2\!\big(1.96/\sqrt{2n_{\mathrm{sm}} - 2}\big)$.

Toroidal mode number. From the cross-phase,

$$\hat{n}(f,t) \;=\; -\frac{180°}{\pi}\,\frac{\arg S_{12}(f,t)}{\Delta\theta}, \tag{1}$$

where $\Delta\theta$ is that pair's toroidal separation in degrees (wrapped positive).

Amplitude, and the $\dot{B} \to B$ conversion. `get_amplitude` with `integrated=True` builds a frequency mask $W(f,t)$ that keeps a bin only if

$$C_{12} \geq 0.975 \quad \text{and} \quad \big|\,|\hat{n}| - |n|\,\big| \leq 0.25, \tag{2}$$

(coherence gate and "integerness" gate, `coh-min` and `eps-int`), weights it by $1/\hat{\omega}^2$ with $\hat{\omega} = \pi k/(n_{\text{fft}}/2)$ and DC forced to zero, and returns

$$A_n(t) \;=\; T_s \sqrt{\text{ff} \sum_f \tfrac{1}{\hat{\omega}^2}\, \tfrac{1}{2}\big(X_{11} + X_{22}\big)} \;=\; \sqrt{\sum_f \Big(\tfrac{1}{\omega}\Big)^2 \tfrac{1}{2}\big(X_{11} + X_{22}\big)\,\text{ff}}, \tag{3}$$

since $\omega = \hat{\omega}/T_s$. This is a frequency-domain time integration: $|B| = |\dot{B}|/\omega$. The result is multiplied by 10^4 to give an RMS $n = 1$ field perturbation in **Gauss**. The normalization $\text{ff} = 2/(p_w \cdot \text{blocksize} \cdot n_{\text{fft}})$, with p_w the window power, is what makes the number an honest RMS.

Robust pair average. The nine per-pair amplitudes are sorted at each time and the two lowest and two highest are dropped (`robust-mean-trim-lo/hi`= 2), leaving the mean of the middle five. This is the defence against a single bad channel or a locally poor coherence.

Triggering. `detect_events` runs `event_detector_` on the $n = 1$ Gauss trace restricted to $[t_A, t_B]$: a Schmitt trigger with *both* levels equal to the threshold (no hysteresis) and an 8 ms debounce on each flank (`debounceDelayTime`). The reported event time is back-dated by the debounce clock, so it is the *crossing* time, not the confirmation time. Only the first positive flank per threshold is retained.

2.3 LM detector: `blmEvents.py`

Different diagnostics, different physics goal: detect a locked or born-locked $n = 1$ island, including the case where it never rotated.

Sensors. Eight saddle loops (`isld*`, radial) and ten midplane probes (`mpid*`, tangential) — the "two-axis" scheme. Pointname sets switch at shot 181137 (`'old'` vs `'new'` naming), which is a maintenance trap worth noting.

Handedness. $\text{sign}(\langle B_t \rangle \langle I_p \rangle)$ selects `RH` or `LH`, which selects between `MmatrixRH.txt` and `MmatrixLH.txt`.

Mode decomposition. The PCS M-matrix (parsed from the pipe-delimited PCS text file) is a least-squares estimator mapping the 18 sensor channels onto 12 targets: sin/cos components of $n = 1, 2, 3$ separated into `in`ternal and `ex`ternal source contributions (`dgsinn1sin`, `dgsexn1cos`, ...). $Z = YM^\top$, and $A^{\text{in}}_{n=1} = 10^4\sqrt{s^2 + c^2}$ Gauss.

Missing channels. If any sensor is unusable, the matrix is *recomputed* rather than patched: `RecoverDesignMatrix` pseudo-inverts M back to the design matrix X via SVD, the corresponding rows of X are deleted, and `ComputeEstimationMatrix` forms $M' = (X^\top X)^{-1} X^\top$ for the surviving subset. A round-trip consistency test (`TestEstimationMatrixHandling`, tolerance 10^{-14}) is asserted at startup. At least `min_sensors`= 14 of 18 must survive.

Preprocessing. `dynamic_rebaselining` subtracts a delayed first-order low-pass estimate of the baseline ($\tau_b = 100$ ms, delay $\tau_d = 100$ ms) and applies a $\tau_f = 5$ ms smoothing — effectively a delayed high-pass that removes slow drift without killing the island signal.

Coil compensation. See Section 7.2; this is the second thing the repository calls calibration.

Triggering. `DetectEventAtLevel` → `auxParseSingleTrace`, same Schmitt-plus-debounce logic, 10 ms delay, over the ladder of `trig_levels`. Both `in` and `ex` branches are tabulated; **only the internal branch is used downstream.**

2.4 Per-threshold base rates

`brmEventsCounter.py` and `blmEventsCounter.py` collapse each ladder into a small table of (threshold, event count, total dwell time, base hazard = counts/dwell). The pair of summary PDFs generated by `mergedEventsExtractPlot.py` shows event count and base rate versus threshold for all four detector variants (RM plain, RM classic, LM compensated, LM uncompensated). This is the sanity check to run first on any new machine: the curve should fall smoothly and show a knee where the detector leaves the noise/sawtooth regime.

3 Stage B — joint labels and the threshold plane

3.1 Joining the two detectors

`eventPathwayAnalysisPlain.py` inner-joins the RM and LM tables on shot, then filters: `isok_rm==1`, `isok_lm==1`, `whichn==1` (RM detection was on $n = 1$), and — crucially — `ta_rm==ta_lm` *and* `tb_rm==tb_lm`. Shots where the two detectors saw different flat-top windows are discarded so that every surviving shot has identical exposure in both channels. Optional cuts on $|\langle I_p \rangle|$ and $|\langle B_t \rangle|$ exist but are unused by the driver.

3.2 The four pathways

`pathway_utils.pathway_type` classifies each shot from $(t_{\mathrm{SOF}}, t_{\mathrm{EOF}}, t_{\mathrm{LM}}, t_{\mathrm{RM}})$:

Condition	Pathway	Markov transitions
neither finite	`uneventful`	PRE→EOF
RM only	`rotating-only`	PRE→RM1→EOF
LM only	`born-locked-only`	PRE→LM1→EOF
$t_{\mathrm{RM}} < t_{\mathrm{LM}}$	`rotating-then-lock`	PRE→RM1→LM2→EOF
$t_{\mathrm{LM}} < t_{\mathrm{RM}}$	`born-locked-then-rotating`	PRE→LM1→RM2→EOF

`OnsetStateAccumulator` accumulates, over the whole database, the dwell time in each of the six states and the counts of each transition, giving a jump matrix J and a rate matrix $R_{ij} = J_{ij}/T_i$. So the framework contains a coarse *continuous-time Markov* description of the RM/LM ordering, independent of any machine learning.

3.3 The threshold-plane scan

For every one of the $73 \times 117 = 8541$ threshold combinations, the accumulator is rebuilt and a profiled log-likelihood is evaluated (`log_likelihood`):

$$\mathcal{L}_s^{\mathrm{exp}} = N_s \left(\ln \frac{N_s}{T_s} - 1 \right), \qquad \mathcal{L}_s^{\mathrm{cat}} = \ln \binom{N_s}{\{J_{s\cdot}\}} + \sum_j J_{sj} \ln \frac{J_{sj}}{N_s}, \tag{4}$$

the first being the exponential-holding-time likelihood at its MLE rate $\hat{\lambda}_s = N_s/T_s$, the second the multinomial likelihood of the branch counts. `checkPathwayPickle.py` sums these over states {PRE, RM1, LM1, LM2} and renders the result — plus pathway fractions, dwell maps and

selected transition rates — as images and contour plots over the (RM threshold, LM threshold) plane, with the operating point marked by `--plot-amp-rm`/`--plot-amp-lm`.

A caveat worth carrying into the MAST-U work. These likelihoods are computed on data that *changes* with the threshold: the event times themselves are threshold-derived. The surface is therefore not a likelihood in the sense that would license a likelihood-ratio test or an argmax, and the code does not treat it as one — nothing auto-selects a threshold. It is a regularity/sharpness map, read together with the pathway-fraction contours, and the actual defence against threshold arbitrariness is the `POINT-A...POINT-F` repetition of the entire study. The README says as much: *"final analysis must study the sensitivity of the results w.r.t. these thresholds"*.

3.4 The joint annotation file

`collect_stats_panda` writes one row per shot:

`shot, tsof, teof, trm, tlm, tevent, ievent`

with $t_{\text{event}} = \min(t_{\text{EOF}}, t_{\text{RM}}, t_{\text{LM}})$ and $i_{\text{event}} = \arg\min$ over that same triple, so

`ievent`	meaning
0	no event; the record is **right-censored** at end of flat-top
1	first event was a rotating mode
2	first event was a locked / born-locked mode

This is a survival table. Only the *first* event matters; everything after t_{event} is discarded.

4 Stage C — EFIT feature harvest

`extractEqDescriptor.py` runs a `toksearch` pipeline over the shots in the joint annotation file, once per EFIT tag. `scratch.create_annotation_index_alt` reads the CSV and sets, per shot, $t_a = t_{\text{sof}}$, $t_b = t_{\text{event}}$ (**not** t_{eof}) and `event`$= i_{\text{event}}$. The descriptor table is then chopped to $[t_a, t_b]$, so each shot contributes an equilibrium time series that ends at its own event or censoring time.

Every feature is deliberately **dimensionless**, normalized by R_0 (`RZERO`, asserted constant) or by a physical scale:

Feature	**Source**	**Definition / note**
`rmin rmax zmin zmax`	`BDRY`, `NBBBS`	boundary bounding box $/R_0$
`aspr`	`AMINOR`	R_0/a (or from bbox with `--use-alts`)
`kappa`	`KAPPA`	elongation
`iphat`	derived	$\mu_0 I_p/(B_{t0}\, a)$ — an inverse-q-like current measure
`nhat`	`DENSITY`	$\bar{n}/n_{\text{GW}}$, $n_{\text{GW}} = \lvert I_p[\text{MA}]\rvert/(\pi a^2)$
`q0 q95`	`Q0 Q95`	safety factor
`elli`	`LI`	internal inductance ℓ_i
`rcur zcur`	`RCUR ZCUR`	current centroid $/R_0$
`betap betat`	`BETAP BETAT`	
`tritop tribot`	`TRITOP TRIBOT`	triangularity
`shaf1 shaf2 shaf3`	`S1 S2 S3`	Shafranov integrals
`glitch`	derived	$\frac{s_1}{4} + \frac{s_2}{4}\left(1 + \frac{R_{\text{cur}}}{R_0}\right) - \left(\beta_p + \frac{\ell_i}{2}\right)$

Feature	Source	Definition / note
`pres0 pres50 dpres50`	`PRES`	p and ∂p at $\psi_N = 0, 0.5$, normalized by $B_{t0}^2/2\mu_0$
`q33 q67 dq67`	`QPSI`	q and ∂q at $\psi_N = 0.33, 0.67$
`error chisq`	`ERROR CHISQ`	*not* model features — retained for screening only

The `glitch` feature. This deserves a note because the README flags it as an open item. The Shafranov relation gives, approximately, $\frac{s_1}{4} + \frac{s_2}{4}(1 + R_{\rm cur}/R_0) \approx \beta_p + \ell_i/2$. The feature is the *residual* of that identity, i.e. a measure of the equilibrium reconstruction's internal self-inconsistency. It is not plasma physics; it is a reconstruction-quality signal — which is exactly why the `NOGL` ablation exists, and why an entry in the README asks for a CER cross check. The `NOGL` versus `FULL` comparison answers whether the model is partly exploiting fit pathology rather than physics.

5 Stage D — the matched EFIT01/EFIT02 merge

This stage is the point of the "matched study". EFIT01 (automatic, magnetics-only) has far better shot and time coverage; EFIT02 needs more diagnostics and is thought better where it exists. A fair comparison requires the two to be evaluated on the same rows.

`processEqDescriptorPair.py --merge` takes EFIT01 as `--primary` and, per shot, for each primary timestamp t_k, searches EFIT02 for any slice within $\pm\delta$ ($\delta = 5$ ms default) and takes the earliest match; unmatched rows become `NaN`. The output (`--save-merge-flattened-all`) is a single flat dictionary:

`X1` (EFIT01), `X2` (EFIT02, matched), `T` (time), `ID` (shot), `Y` (label), `Xnames`, `tag1`, `tag2`

with `X1` and `X2` sharing the same shape, row indexing, and column names by construction.

Label placement. In `table_stack`:

```
if T12[i]["event"] > 0:
    if T12[i]["tb"] - ith_tend <= event_delta:   # event_delta = 40 ms
        Y[r + ni - 1] = T12[i]["event"]
    else:
        lost_events += 1
```

So **exactly one row per event shot carries a label, the last one**, and only if the last available EFIT slice is within 40 ms of the event time. If the EFIT grid ran out earlier, the event is *dropped* and counted in `lost_events` — a printed diagnostic worth watching. Censored shots contribute all-zero rows. `Y` temporarily holds the event *type* $(1, 2)$; `prep_regression_tables` later binarizes it, so the trained model pools the two competing risks into a single "TM onset" event. The type information survives in the tables and could be used for a competing-risks extension.

Screening. `apply_double_screening` (`tabular_limits.py`) requires a row to pass limits in *both* EFITs — $\chi^2 \le 90$, `error` $\le 5 \times 10^{-3}$, $0 \le \hat{n} \le 2$, $\beta_p, \beta_t > 0$, $0 < q_0 \le 10$, $1 < q_{95} \le 20$, plus generous bounds on the pressure features. `chisq` and `error` are then removed from the feature list. The `--limits auto` alternative derives limits from the 0.05%/99.95% quantiles of each column instead.

6 Stage E — the boosted-tree hazard model

6.1 The objective is a survival likelihood, not a classifier

`xgb_hazard_common.py` supplies a custom XGBoost objective. With f the raw margin and $p = \sigma(f) = (1 + e^{-f})^{-1}$,

$$\ell(y, f) = -y \ln p - (1 - y) \ln(1 - p), \qquad g = p - y, \qquad h = p(1 - p), \tag{5}$$

which is ordinary logistic loss — but applied to *this* table it is exactly the discrete-time survival log-likelihood with right censoring. A shot observed over slices $k = 1 \dots K$ contributes

$$\underbrace{\prod_{k=1}^{K-1} (1 - p_k) \cdot p_K}_{\text{event at } K} \qquad \text{or} \qquad \underbrace{\prod_{k=1}^{K} (1 - p_k)}_{\text{censored}}, \tag{6}$$

and these are precisely the products of Bernoulli terms produced by the one-1-at-the-end labeling. The custom objective is registered rather than using `binary:logistic` so that the reported metric is ⟨nllikl⟩ in nats/sample, comparable across runs and directly against a base-rate model.

6.2 Base-rate anchoring

Before every fit,

$$\texttt{base\_score} = \operatorname{logit}\left(\frac{\sum_i y_i}{N}\right) \tag{7}$$

computed *on the training partition only.* Two consequences: the boosting starts from the correct event rate rather than from 0, and $f = f_{\text{base}}$ acquires the meaning "no information beyond the population base rate". Every log-odds figure in the codebase draws that line.

6.3 Shot-respecting resampling

All splitting goes through the shot key `S.` `make_xgb_cv_folds` partitions *unique shots* K-ways, and `outer_fold_test_preds` asserts the train/test shot-number intersection is empty. Since consecutive EFIT slices within a shot are strongly correlated, a naive row-wise split would leak badly and inflate performance. The nesting is:

Outer loop	`test-fold-K` shot-disjoint folds, repeated `num-K-samples` times
Inner loop	`cross-validation-k`-fold `xgb.cv` for early stopping (patience 25)
Rounds	`max-rounds` = 1000, best round = arg min of `test-<nllikl>-mean`

The ensemble suite uses 8 outer folds × 10 repeats; the held-out-prediction suite uses 12 outer folds × 5 repeats with SHAP. Hyperparameters (both drivers): `max_depth`= 6, `eta`= 0.10, `lambda`= 0.01, `min_child_weight`= 5, `tree_method=hist`, `max_bin`= 512.

6.4 What comes out

`outer_fold_test_preds` returns an $N \times B$ matrix `PREDS` in which *every row is a genuinely out-of-sample log-odds*, from a model that never saw that shot, with B independent repeats providing a spread. This matrix is the input to the calibration diagnostic and to every per-shot trace plot. Also saved: the fitted booster ensemble with its training shot keys (`--save-model-ensemble`), per-row SHAP contributions, and the pooled losses against the base-rate loss.

6.5 Two further model-inspection machines

- **Elimination paths** (`elimination_path_outer_sample`): repeatedly train, log held-out loss, drop the least important feature, repeat to exhaustion — once per outer fold. No decision is taken on test performance, so the resulting loss-versus-feature-count curve is a descriptive object. Inspect with `python xgbShowElimination.py --folder <folder>`. The README notes an unresolved puzzle: the shuffle-control path is suspiciously orderly, likely a random-tie-breaking artifact in `argmin` over importances.
- **Feature linkage** (`feature_distance_matrix`): fit n_x univariate models, then try to predict each univariate model's output from every *other* single feature, and score with KL divergence in both directions. Small distance ⇒ redundancy. This is the intended answer to "which of my 20 features are really the same feature". Presentation is still a README TODO.

7 Calibration — what it is, what it does, what it is for

The word *calibration* is used in this repository for two entirely unrelated operations. Both are worth knowing, but only the first is what the modeling discussion refers to.

7.1 Hazard calibration (the statistical one)

Where: `plot_calibration` in `xgbDescriptorModel.py` (flag `--full-table-model-calib`); standalone in `testcalib.py`; Bayesian version in `test_beta_prior.py`; older sibling in `evalGBM.py`.

The problem it solves

The trained booster emits a dimensionless margin $f(x)$. A log-odds is not a physical quantity: it is defined only relative to the sampling interval used to build the labels. What a hazard study actually needs is a **rate** $h(x)$ in s^{-1} — something that can be integrated into a survival curve, quoted as an expected time-to-onset, or compared between machines. The calibration step is the bridge, and the bridge is one identity.

The identity

Each table row represents one EFIT slice covering a time interval Δ. The model's p is the probability that onset happens *in that interval*. For a constant hazard h across the interval,

$$p \;=\; 1 - e^{-h\Delta} \qquad \Longrightarrow \qquad h\Delta \;=\; -\ln(1-p) \;=\; -\ln\!\left(1 - \frac{1}{1+e^{-f}}\right), \tag{8}$$

$$\boxed{h\Delta \;=\; \ln\!\left(1+e^{f}\right) \;=\; \mathrm{softplus}(f)} \tag{9}$$

So the softplus of the model margin *is* the dimensionless hazard Δh, and

$$h(x) = \frac{\mathrm{softplus}\left(f(x)\right)}{\Delta}\ \ [\mathrm{s}^{-1}], \qquad H(t) = \sum_{k\,:\,t_k \le t} \mathrm{softplus}(f_k), \qquad S(t) = e^{-H(t)}, \tag{10}$$

$$\text{hazard ratio} \;=\; \frac{\mathrm{softplus}\left(f(x)\right)}{\mathrm{softplus}\left(f_{\mathrm{base}}\right)}, \qquad f_{\mathrm{base}} = \mathrm{logit}\!\left(\frac{E}{N}\right). \tag{11}$$

Every one of these appears in `testcalib.py`: `hazhat = np.log(1 + np.exp(FS))`, `cumh = np.cumsum(hazhat)`, the `hazhat/H0` ratio panel, and the per-shot CSV columns `avgh`, `avghr`, `endhr`. The author's own scale reference in the comments: "*baserate means that the expected dwell time is* ~ 10 *seconds*" — consistent with $E/N \sim 2 \times 10^{-3}$ at $\Delta \approx 20\,\mathrm{ms}$.

The plot, and how to read it

The calibration figure is a reliability diagram, but drawn in hazard units against the raw margin:

1. Bin the pooled held-out margins f into `nbins`= 40 bins spanning the $[1\%, 99\%]$ quantile range of f (the tails are cut because they hold too few samples for a rate estimate).
2. Per bin, count all samples C and event samples E. Repeat for each of the B prediction replicas, giving B curves.
3. Plot $\overline{E/C}$ against bin centre, and overlay the reference curve $y = \ln(1 + e^f)$.
4. Mark the base rate as a horizontal line $h_0 = \text{softplus}(f_{\text{base}})$ and a vertical line at f_{base}.

The empirical E/C in a bin estimates the interval event probability p, whereas the reference line is $-\ln(1-p)$. These agree to $O(p^2)$, and with $p \sim 2 \times 10^{-3}$ the difference is invisible — which is exactly why the same curve can be labelled "ideal calibration" and the y-axis labelled "normalized hazard $\Delta \cdot h$" without contradiction. A companion two-axis figure shows the raw sample and event counts per bin, so you can see *where* the statistics support the claim.

Diagnosis:

- **Points on the line** — the score is a calibrated hazard. You may quote $h = \text{softplus}(f)/\Delta$ in s^{-1}, integrate it, and use hazard ratios.
- **Points systematically flatter than the line** — the model is over-confident at the extremes: high-f samples do not fail as often as claimed. Typically over-fitting, or leakage from a train/test split that did not respect shots.
- **Points steeper** — under-confident; the model is shrinking too hard toward the base rate.
- **Curve crosses the base-rate lines away from their intersection** — the base-rate anchoring is off, e.g. the screening changed E/N after `base_score` was set.
- **Wide spread across the B replicas in a bin** — that region of feature space is not resolved by the data volume; do not quote a rate there.

Three things it is *not*

1. **It is not a fitting step.** There is no Platt scaling and no isotonic regression anywhere in this repository. Nothing is refit, rescaled, or corrected. Calibration here is purely a *verification* that the raw margins can be read as rates. If the curve missed the reference line, the response would be to change the model or the labeling, not to post-hoc warp the output. This is the single most common misreading of the term, and the reason the code says "calibratedness".

2. **It is not an in-sample plot in the runs that matter.** The honest version consumes `PREDS` from `outer_fold_test_preds`: every margin comes from a model trained without that shot. The `--full-table-model --full-table-model-calib` combination *does* produce an in-sample version, which is useful for debugging and optimistic by construction.

3. **It is not a discrimination metric.** A model can be perfectly calibrated and useless (predict the base rate everywhere — it sits at a single point on the line). Calibration and sharpness are separate axes: the loss comparison `avg_llikl` versus `avg_llikl_0` carries the sharpness claim, the calibration plot carries the interpretability claim.

The Δ dependence — the part that bites when porting

Equation (9) contains Δ. The consequences are easy to get wrong:

- The labels, the base rate E/N, and hence the entire f scale are properties of the **EFIT slice cadence**. Halving Δ roughly halves E/N and shifts every f by $\approx \ln 2$.
- Therefore f **is not portable** across machines, EFIT variants, or time bases. Only $h = \text{softplus}(f)/\Delta$ is, and only if Δ is genuinely uniform.
- `check_data_status` prints `np.median(np.diff(data["T"]))` for exactly this reason — confirm Δ before quoting any rate. Note that this median is taken across shot boundaries in the flattened table; it is a diagnostic, not a rigorous per-shot cadence check.
- The framework assumes a fixed Δ. A non-uniform time grid would require an explicit exposure/offset term in the objective (a Poisson-style $\ln \Delta_i$ offset), which is not implemented. Worth checking before adopting a variable-cadence equilibrium code.

7.2 Magnetics calibration / compensation (the hardware one)

The same word appears in the detector layer with a completely different meaning: removing known non-plasma contributions and putting probe signals on an absolute physical scale.

Coil pickup compensation (LM detector). `ExtractCompensationCoefficients` reads `cm_2018.sav`, an IDL save file holding a coil×probe matrix `cm` of mutual coupling coefficients. With `--dcccoils --dcicoils` the driver requests compensation for the six C-coils and twelve I-coils, and the pipeline subtracts the predicted pickup before any mode fitting:

```
X = X - np.dot(time_rebase_columns(coil_t, coil_X[:, ok], t), compMatrix)
```

Without this, deliberately applied 3D fields (error-field correction, RMP) appear in the sensors as an apparent $n = 1$ island and trigger the LM detector. The driver deliberately produces *both* versions — `fdp-regen-blm-nic-*` (uncompensated) and `fdp-regen-blm-ic-*` (compensated) — and the study consumes the compensated one. Comparing the two base-rate-versus-threshold curves quantifies how much of the raw LM count was coil pickup. The internal/external M-matrix separation is a second, independent line of defence against the same contamination, which the file header concedes works "imperfectly"; `checkCompensation.R` and `magneticsCheckPCS.R` exist to audit this.

Absolute amplitude scale (RM detector). The $1/\omega^2$ spectral weighting plus T_s factor and the 10^4 multiplier are what make the RM threshold ladder a ladder in **Gauss of RMS n=1 field at the probes** rather than in arbitrary $\dot{B}$ units. The ff normalization carries the window-power and FFT-length bookkeeping. If any of these are wrong the detector still works, but the thresholds lose their physical meaning and cannot be compared to published DIII-D numbers. Note also the caution in `test-mpxspec.py` about a possible scaling difference between PCS and standard-archive versions of the same signal.

7.3 Summary answer

	Hazard calibration	**Magnetics compensation**
What	Check that $\overline{E/C}$ per margin bin follows $\ln(1+e^f)$	Subtract modelled coil pickup; scale $\dot{B}$ spectra to absolute Gauss
Where	`plot_calibration`, `testcalib.py`	`blmEvents.py` + `cm_2018.sav`; `modespyec.get_amplitude`
Does	Licenses reading the score as $h = \text{softplus}(f)/\Delta$	Removes non-plasma signal; fixes threshold units
For	Survival curves, expected time-to-onset, hazard ratios, cross-run comparison	Preventing applied-field triggers; making thresholds physically meaningful
Applied?	**No** — diagnostic only, nothing is refit	**Yes** — it modifies the data

8 Porting to MAST-U

8.1 What transfers unchanged

The architecture, and it is the valuable part: threshold-ladder detectors; a joint annotation file as the single interface between detection and modeling; right-censored one-label-per-shot survival tables; logistic loss as the survival likelihood; base-rate anchoring; shot-disjoint nested resampling; and the softplus hazard link with its calibration check. None of this is DIII-D specific. Files `pathway_utils.py`, `xgb_hazard_common.py`, `tabular_limits.py`, and the `plot_calibration` routine are essentially machine-agnostic.

8.2 RM detector: the concrete change list

Working outward from `modespyec.py`, in rough order of how much damage a wrong choice does:

1. **Probe list and toroidal angles** are hard-coded in `brm_events_modespyec.py`'s `__main__`, together with the `'d'` pointname suffix. Replace with a machine configuration. Note two structural assumptions in `summarize_consecutive_circular_pairs`: the outboard angles must be sorted ascending (asserted), and the last channel is expected to be an `mpi11m*` probe that is required present but never used in the pairing — with $N{+}1$ input channels only the first N enter the N wrapped consecutive pairs. Understand that before reusing the function.

2. **Sample-rate gate.** `okTs` hard-asserts $5\,\mu$s $\pm 0.1\%$. This must become a parameter, and any resampling must happen before the FFT stage.

3. **Window duration, not window length.** `blocksize`= 1024 at 200 kHz is a 5.12 ms window with 2.56 ms stride. Preserve the *duration* for a given MAST-U digitizer rate, not the sample count; this sets the frequency resolution $\Delta f \approx 195$ Hz and therefore what counts as a resolvable rotating mode. MAST-U mode frequencies and pulse lengths differ from DIII-D, so this is a physics choice.

4. **Threshold ladder range.** 1–30 G is set by DIII-D probe geometry and field levels. Rederive the range from your own base-rate-versus-threshold curve (`brmEventsCounter.py` → `mergedEventsExtractPlot.py`) and place the ladder to bracket the knee. Do not import 5.0 G.

5. **Coherence and integerness gates.** `coh-min`= 0.975 and `eps-int`= 0.25 were tuned against DIII-D noise and a nine-pair array. With fewer probes or wider spacing the $\hat{n}$ estimate degrades and `eps-int` must widen — at the cost of n-aliasing. Also check $\Delta\theta$ against the Nyquist limit in toroidal mode number for your array.

6. **Robust trimming.** `trim-lo/hi`= 2 assumes nine pairs (mean of the middle five). With a smaller array this discards too much; scale it or switch to a median.

7. **Debounce.** 8 ms (RM) and 10 ms (LM) relative to DIII-D flat-tops of seconds. MAST-U pulses are shorter; re-express the debounce as a fraction of the analysis window or of the expected growth time.

8. **Flat-top windowing.** `iptimestamp` thresholds — $t_A > 0.40\,\mathrm{s}$, $t_B > 1.00\,\mathrm{s}$, $|\langle I_p \rangle| > 0.50\,\mathrm{MA}$, the 500 ms smoothing boxcar, and the `t2ratio < 1.05` disruption heuristic — are all DIII-D magnitudes and will silently reject most MAST-U pulses. This is the most likely cause of an unexpectedly empty results table, so validate it first, in isolation, on a handful of known shots.

9. **Sawtooth/ELM rejection.** The "plain" detector's whole premise is that a high enough Gauss threshold excludes sawteeth without parity discrimination. That premise needs re-testing on MAST-U, where the ELM and fishbone signatures in midplane $\dot{B}$ differ. If the base-rate curve shows no clean knee, the older even/odd-parity approach in `mpxspec.py` becomes relevant again.

Practical suggestion: `show_brm_detectors.py` and `show_all_detectors.py` exist to overlay detector variants on individual shots. Getting a MAST-U equivalent of those working on a dozen hand-picked pulses, *before* launching any database-scale run, is the highest-value first step.

8.3 Downstream considerations

- **No LM detector without an equivalent decomposition.** The LM branch depends on a PCS M-matrix and a saddle-loop/probe two-axis array. Absent a MAST-U analogue, the joint annotation degenerates to RM-only — which `pathway_utils` handles gracefully (`rotating-only` and `uneventful` pathways), but the pathway analysis and threshold plane collapse to one dimension. `eventPathwayAnalysisPlain.py` as written requires both CSVs and asserts on both; that is the code to adapt.

- **Equilibrium features.** MAST-U's reconstruction (EFIT++) exposes different node names and, importantly, a different slice cadence Δ — see the Δ discussion in Section 7. The dimensionless normalization scheme ($/R_0$, $\mu_0 I_p/B_{t0}a$, $\bar{n}/n_{\mathrm{GW}}$, $p/(B_{t0}^2/2\mu_0)$) is designed to be machine-transferable and should be kept. Spherical-tokamak aspect ratio pushes `aspr` well outside the DIII-D range, so any `--limits fixed` screening bounds must be re-derived; `--limits auto` (quantile-based) is the safer starting point.

- **The matched-pair machinery is optional.** `processEqDescriptorPair.py` exists to compare two EFIT variants. With a single reconstruction, pass the same pickle as primary and secondary, or write a thinner flattener — but keep `table_stack`'s 40 ms event-proximity guard and its `lost_events` count, which is a real data-quality signal.

- **Event counts drive everything.** With B replicas over K shot-disjoint folds and a base rate of order 10^{-3}, the usable resolution of the calibration plot is set by the total event count, not the row count. Estimate E early; it determines whether 40 bins are defensible or whether the diagnostic must be coarsened.

9 The MAST-U implementation: audit and retrofit

Section 8 is the generic change list. This section is an audit of the MAST-U code as it actually stands (`~/Dropbox/Python/MASTu`, with E. Giovannozzi's analysis modules in `~/Dropbox/Python/Hazard/egio`), together with the retrofit applied to it.

9.1 Two mechanisms, first

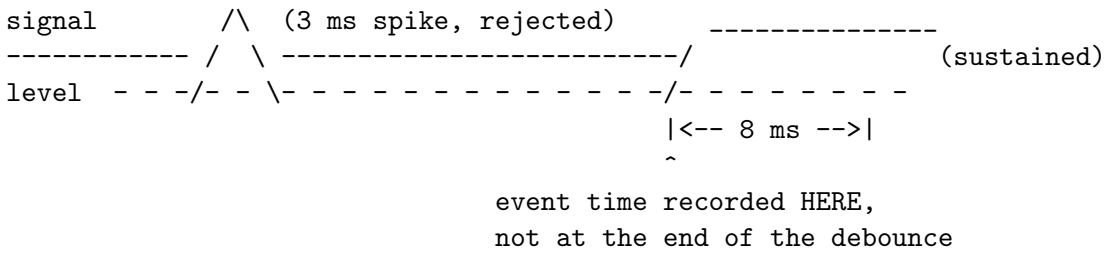
Both appear repeatedly above and both are load-bearing, so they are set out plainly here.

9.1.1 The Schmitt trigger and its debounce

A plain comparator — "is the signal above the level?" — *chatters*. When the signal sits near the threshold, noise flips the answer back and forth and a single physical excursion is reported as dozens of events. A **Schmitt trigger** is a comparator with memory. In its classical form it carries two levels: the output switches on at a high level and off at a lower one, so once triggered it cannot drop out until the signal has fallen well back. The gap between the two levels is called *hysteresis*.

The DIII-D detector (`event_detector_` in `modespyec.py`) sets both levels equal, so there is no hysteresis, and suppresses chatter a different way: with a **debounce**. The signal must remain above the level *continuously* for τ_d before an event is declared — 8 ms for the RM detector, 10 ms for the LM detector. A shorter excursion resets the clock and is discarded.

The detail that matters is what happens next. Once confirmed, the reported event time is *back-dated* by τ_d, so the record holds the moment of first crossing, not the moment of confirmation:



The physical justification is that a tearing mode grows and persists, whereas a sawtooth crash, an ELM, or a digitizer glitch does not. Without the debounce the "event time" is the arrival time of the first noise spike above threshold, which is a property of the noise, not of the plasma. Without the back-dating, every event time carries a fixed $+\tau_d$ bias — harmless for a classifier, but not for a survival model whose whole content is *when* things happen.

9.1.2 The coherence gate

A detector array sees a superposition of a coherent mode and incoherent noise. The coherence gate is the test that separates them, and it is what licenses calling an amplitude a mode amplitude.

Take N_c sensors at toroidal angles φ_c, and let $S_c(f,t)$ be the windowed Fourier transform of sensor c. A genuine mode of toroidal number n appears at every sensor with the same amplitude and a phase fixed by position, $S_c \propto e^{in\varphi_c}$. Removing that expected phase pattern and summing,

$$A_n(f,t) \;=\; \sum_{c=1}^{N_c} e^{-in\varphi_c}\, S_c(f,t), \tag{12}$$

gives two very different behaviours:

- **Coherent mode present.** Every term rotates into alignment and the sum adds constructively: $|A_n|^2 = N_c^2|S|^2$.
- **Uncorrelated noise.** The phases are random, the sum is a random walk, and it grows only as $\sqrt{N_c}$: $\mathbb{E}|A_n|^2 \approx N_c\langle|S_c|^2\rangle$.

Normalising by the best case gives a dimensionless quality factor on $[0, 1]$,

$$\gamma_n^2(f,t) \;=\; \frac{|A_n(f,t)|^2}{N_c^2\,\big\langle|S_c(f,t)|^2\big\rangle_c} \qquad\longrightarrow\qquad \begin{cases} 1 & \text{perfect } n\text{-mode,} \\ 1/N_c & \text{pure noise.} \end{cases} \tag{13}$$

This is the fraction of array power explained by that toroidal harmonic — the difference between a crowd shouting a word in unison and the same crowd shouting at random.

A **gate** is then simply a cutoff: any (f, t) bin with γ_n^2 below the threshold contributes nothing to the amplitude sum. DIII-D applies two such gates together (Section 2.2): magnitude-squared coherence $C_{12} \geq 0.975$ between probe pairs, *and* an integerness condition $\big|\,|\hat{n}| - |n|\,\big| \leq 0.25$ requiring the fitted mode number to be close to a whole number.

The consequence of omitting the gate is not subtle. The mode number is assigned by an arg max over candidate n, and an arg max always returns something. Every noise bin is therefore labelled with some n and admitted to the amplitude sum, and the resulting "mode amplitude" tracks the noise floor.

9.2 Code inventory

File	Role and state
`run_master.py`	Intended top-level driver
`saddle_analysis.py`	OMAHA batch analysis; the source of `saddle_analysis_one_shot`
`run_saddle_analysis.py`	Earlier scratch version; contains a `load_ohama_slow` typo and mixed tabs/spaces, i.e. it does not parse. Dead.
`flattop.py`	*New* — flat-top windowing
`mag_perturbations_shots47000_47099.pkl`	78 shots of collected output
`egio/saddle_data.py`	Giovannozzi's MAST-U magnetics library: OMAHA loading, spectrogram, n-detection, amplitude extraction
`egio/model_saddle.py`, `egio/lazytraits.py`	Present but not on this path

Five of Giovannozzi's modules are Freya-only and absent locally: `omaha_coils` (the coil table: toroidal angle, polarity, orientation, slow/fast channel names), `saddle_geometry`, `mode_functions` (`zeros_spectrum`), `pickup_coil_data`, `sxr_geometry`. `omaha_coils` is the important one: it is the array geometry, and therefore the MAST-U counterpart of DIII-D's hard-coded probe angle list.

9.3 Empirical finding: the detector currently follows the noise floor

The collected pickle holds 78 shots (of 100 attempted, 47000–47099), each with 779 time slices per toroidal mode number, $n \in \{1, 2, 3, 4\}$. Reading it back gives an unusually clean diagnosis.

Time base. Identical across all 78 shots: $t_0 = 1.122\,\text{ms}$, $t_{\text{end}} = 996.962\,\text{ms}$, $\Delta t = 1.28\,\text{ms}$ exactly, 779 points. From $\Delta t = \texttt{NFFT}/2$ samples with $\texttt{NFFT} = 512$, the OMAHA *slow* channels digitise at

$$F_s = 200\ \text{kHz}, \qquad \text{window} = 2.56\ \text{ms}, \qquad \text{stride} = 1.28\ \text{ms}, \qquad \Delta f = 391\ \text{Hz}. \tag{14}$$

This is the same sample rate as the DIII-D midplane probes, so the window-duration reasoning of Section 8 transfers directly. Note that $\texttt{NFFT} = 512$ gives exactly *half* the DIII-D analysis window; $\texttt{NFFT} = 1024$ would match it.

The detector does not distinguish plasma from vacuum. Comparing the $n = 1$ amplitude before breakdown against the same quantity during the flat-top, pooled over all 78 shots:

$n = 1$ amplitude, $t < 50\,\text{ms}$ (no plasma)	1.92×10^{-7}
$n = 1$ amplitude, $0.2 < t < 0.8\,\text{s}$ (flat-top)	2.52×10^{-7}
ratio	1.31

A 31% difference. Supporting evidence points the same way: `freq` is finite on 100% of slices with a median of 14.2 kHz, i.e. a mode is reported at every instant of every shot including the vacuum phase, and the value is close to the centroid of the $[0.1, 50]\,\text{kHz}$ search band — the signature of a broadband noise spectrum, not a mode.

Cause. In `saddle_data.py`, `Spectra.n_detection` computes precisely the coherence of Eq. (13),

```
coherence = np.abs(ap_sp) ** 2 / self.phi.size ** 2 / power
```

and `RecognizedModes.threshold()` exists to gate on it. But `RecognizedModes.amplitude()` never calls either. The only mask applied is `Ntor != sel.ntor`, which is the arg max assignment — and the one line that could have rejected weak samples,

```
Fa[Bp < amplitude_selector.v_min] = np.nan
```

is inert because the driver passes `v_min` = 0.0. Nothing is ever rejected. The pipeline as it stands is a spectral summary, not a detector.

Consequence for the locked-mode labels. `LM_labels = (freq < 2000)` fires on 0.44% of $n = 1$ slices (0.06%, 0.04%, 0.19% for $n = 2, 3, 4$), with no amplitude and no coherence condition. These are slices where the noise centroid happened to dip below 2 kHz. There is a second, deeper problem: the amplitude search band starts at 100 Hz, so a genuinely locked mode — for which $f \to 0$ — falls *below* the band and vanishes rather than being labelled. DIII-D uses an entirely separate, non-spectral path for locked modes (the M-matrix decomposition of Section 2.3) for exactly this reason. A frequency cut on a spectral detector is not a substitute for a locked-mode detector.

9.4 Structural defects in the driver

Independently of the physics, `run_master.py` could not run:

1. `saddle_analysis.py` executed its 100-shot `pyuda` scrape *at module level*, so `from saddle_analysis import saddle_analysis_one_shot` triggered a full database sweep and a pickle write on import.
2. `run_master` called `build_rm_detector()` and `build_client()`; neither exists anywhere in the codebase.

3. `collect_signals` read `n_tor_max`, `NFFT`, `SaddleFeats` and `client` as globals that were never in scope, and returned bare `None`. It was never called.
4. `ntor = np.arange(n_tor_max)` starts at 0. The $n = 0$ component is axisymmetric and carries no mode information.
5. `saddle_analysis_one_shot` read `time_interval`, `freq_min` and `freq_max` as module globals. Their values differ between the two scripts ($f_{\min} = 100\,\mathrm{Hz}$ versus $1\,\mathrm{kHz}$), which silently changes what counts as locked.
6. The batch loop caught every exception and `passed`, so the 22 missing shots carry no record of why they were dropped.

Two further bugs live in `saddle_data.load_omaha` and `load_base` and should be fixed at the source: the `except` clause resets `time` and `data` to empty arrays, so a single failing channel destroys everything already loaded; and `idt = np.flatnonzero(np.diff(time) < 1e-4)` is then used only as `time[idt[0]:idt[-1]]`, which discards the mask and takes a contiguous slice instead. The commented-out uniform-Δt assertion in `SaddleFull.spectrum` suggests these were workarounds for irregular time bases — worth understanding before trusting the FFT stage.

9.5 The retrofit

`flattop.py` **(new).** The `iptimestamp.py` analogue, which had been a `# timeFT = find_flattop(shot, Ip)` placeholder. It reproduces the DIII-D four-timestamp construction — smooth I_p, differentiate, locate the extreme ramp rates, back-track to where the rate falls below α of the extremum — with every machine constant factored into a `FlatTopConfig` dataclass (30 ms smoothing rather than 500 ms, $|\langle I_p \rangle| > 50\,\mathrm{kA}$ rather than 0.5 MA, and so on). It returns the window together with the flat-top quality metrics (linear-fit slope, normalised RMSE) and, on failure, a string saying which test rejected the shot. The DIII-D disruption heuristic (`t2ratio`) is carried over.

`saddle_analysis.py` **(repaired, not rewritten).** The original routine with four defects fixed and nothing else touched. The signature `saddle_analysis_one_shot(shot, NFFT, ntor, SaddleDict)` is unchanged, the output keys are unchanged, and the pickle layout `{shot: {n: {...}}}` is unchanged, so existing readers and existing `.pkl` files keep working. The four repairs:

1. the batch driver moves under `if __name__ == "__main__"`, so importing the module no longer triggers a database sweep;
2. `time_interval`, `freq_min` and `freq_max` become optional keyword arguments *after* the original four positionals — they were module globals read from inside the function, so any import-and-call raised `NameError`;
3. drop reasons are written to a `.dropped.json` sidecar rather than swallowed by `except: pass`, leaving the pickle format untouched;
4. a dead `amp = modes[itor].amplitude` is removed.

`saddle_extras.py` **(new).** Everything genuinely new lives here, so that the original routine above stays independently testable. It provides `amplitude_with_coherence()`, which mirrors Giovannozzi's `amplitude()` exactly for amplitude and frequency but additionally reduces `rm.coherence` over the same time/frequency selection and n-mask,

$$\bar{\gamma}_n^2(t) \;=\; \frac{\sum_f \gamma_n^2(f,t)\, P(f,t)}{\sum_f P(f,t)} \qquad \text{(power-weighted over the surviving bins)}, \tag{15}$$

so that *every amplitude sample now carries the quality of the toroidal fit that produced it* and the gate of Section 9.3 can be applied downstream. It also provides `noise_floor()` and the enriched per-shot entry point `analyze_shot()`, which returns `(per_n, meta)` as a tuple rather than mixing a string key into an integer-keyed dictionary.

`run_master.py` **(reworked).** `collect_signals()` becomes the real per-shot entry point: it fetches I_p, derives the flat-top window, runs the OMAHA analysis, builds a normalised and coherence-gated detector trace per n, and gathers the auxiliary signals. `build_rm_detector()` then runs the threshold ladder over a shot list and writes the ladder table — the direct analogue of `fdp-regen-brm-plain-*.csv` — with one column per level holding the first crossing time and `NaN` where the level was never crossed. The trigger, `schmitt_first_crossing()`, implements the debounce and back-dating described above.

The per-shot noise floor. MAST-U offers something DIII-D does not. The OMAHA record opens at $\approx 1\,\mathrm{ms}$, well before breakdown, so every shot carries ~ 39 slices of genuine no-plasma baseline. This matters because the per-shot median amplitude varies by a factor of 9.7 across the 78 shots — a fixed absolute ladder would be dominated by shot-to-shot variation in the noise floor rather than by plasma behaviour. The ladder is therefore currently defined in units of each shot's own pre-plasma floor,

$$\mathrm{SNR}(t) = \frac{A_n(t)}{\mathrm{median}\left(A_n(t < 20\,\mathrm{ms})\right)}, \tag{16}$$

which should be replaced by an absolute ladder once the calibration below is settled.

9.6 Offline validation, and a handedness trap

`amplitude_with_coherence()` duplicates roughly twenty-five lines of upstream code, which is the kind of change that should not be trusted on a reading. It was therefore exercised offline against Giovannozzi's *actual* `SaddleFull`, `Spectra` and `RecognizedModes` classes, with the Freya-only modules stubbed, on synthetic data: eight coils at 200 kHz, white noise throughout, and an $n = 2$ mode at 10 kHz switched on at $t = 0.15\,\mathrm{s}$. Three things were confirmed:

- `rm.coherence` has shape $(n_{\mathrm{searched}}, n_f, n_t)$, so the index expression `rm.coherence[i].T[:, idf][idt, :]` is correct;
- the returned amplitude and frequency are **bit-identical** to `RecognizedModes.amplitude()` for every n;
- in the mode window the coherence for the injected n reaches 0.9994 against a measured noise floor of ≈ 0.25, while every other n stays at the floor.

Two numbers from that test are worth keeping. The measured pure-noise coherence floor sat near $2/N_c$ rather than the theoretical $1/N_c$, so a gate should be set from the measured distribution and not from theory; with a floor of 0.25 and a genuine mode near 1.0, anything in 0.5–0.7 separates cleanly.

The handedness trap. The first run of this test produced the coherence *backwards* — it fell in the mode window instead of rising. The cause is a phase convention. `Spectra.n_detection` builds the projection

```
ap = np.exp(1j * np.radians(phi) * ntor[:, None])
```

that is, it projects onto $e^{+in\varphi}$ and therefore detects modes whose spectral phase runs as $e^{-in\varphi}$. A mode of the opposite handedness is invisible to a positive-only search list. Repeating the test with the sign of the injected toroidal phase flipped:

injected mode		coherence at $n = 2$	amplitude ratio (mode/noise)
$\sin(\omega t - n\varphi)$	(matches convention)	0.9994	99.5
$\sin(\omega t + n\varphi)$	(opposite)	0.0092	13.4

The second row is the important one. The coherence correctly collapses — but the *ungated* amplitude still rises by a factor of 10–17 for **every** n simultaneously. A mode of the wrong handedness does not merely go undetected: it inflates the reported amplitude of all four toroidal numbers at once. That is exactly the false positive the gate exists to remove, and it is a second, independent route to the noise-following behaviour of Section 9.3.

Which handedness a real mode carries depends on the toroidal rotation direction and hence on the signs of I_p and B_t. DIII-D handles this explicitly: the locked-mode detector branches on $\mathrm{sign}(\langle B_t\rangle\langle I_p\rangle)$ to select `MmatrixRH` or `MmatrixLH` (Section 2.3). The MAST-U path has no such branch and `ntor = [1,2,3,4]` is one-sided. Until the convention has been checked against a shot with a known mode, both signs should be searched — `saddle_extras.NTOR_BOTH_SIGNS` — and the half carrying the coherence identified. If the campaign runs a single I_p/B_t polarity this reduces to a one-off determination; if both polarities appear in the database, it must be resolved per shot, exactly as DIII-D does.

9.7 The workflow as it now stands

All modules live in `~/Dropbox/Python/Hazard`, alongside `egio/saddle_data.py`. The call graph for one shot:

```
run_master.main()
  argparse --> run_master(shots, build_detector, thresholds, ...)
    pyuda.Client()
    build_rm_detector(shots, client, ladder, ...)
      |
      for each shot:
      |
      +-- collect_signals(shot, client, ...)          [run_master.py]
      |     |
      |     +-- client.get("/AMC/PLASMA_CURRENT")     measured Ip, kA -> MA
      |     +-- flattop.find_flattop(t, ip, cfg)      --> FlatTop(ok, t_a, t_b,
      |     |                                                ip_mean, slope, nrmse,
      |     |                                                reason)
      |     |     drops the shot if any window test fails
      |     |
      |     +-- saddle_extras.analyze_shot(shot, NFFT, ntor)
      |     |     +-- saddle_data.load_omaha_slow(shot)      --> SaddleFull
      |     |     +-- SaddleFull.spectrum(NFFT)              --> Spectra
      |     |     +-- Spectra.n_detection(ntor)              --> RecognizedModes
      |     |     +-- for each n:
      |     |           saddle_extras.amplitude_with_coherence(rm, sel)
      |     |             --> (time, freq, Bp, Ba, coh)
      |     |           saddle_extras.noise_floor(time, Ba, t_pre)
      |     |     returns (per_n, meta)
      |     |
      |     +-- build detector trace per n:
      |     |       snr = amp / noise_floor
      |     |       snr[coherence < coherence_min] = NaN      <-- the gate
      |     |       restrict to [t_a, t_b]
      |     |
      |     +-- _grab(): XIM/DA/HM10/T, ANE/DENSITY (+dne_dt), AYC/T_E
      |     |
      |     returns {ok, reason, tsof, teof, ip_mean, ft_nrmse,
      |              traces{n: {time, snr, coherence, floor}}, modes, meta, aux}
      |
      +-- detect_ladder(sig, ladder, n_detect, debounce)
            for each level:
```

```
            schmitt_first_crossing(time, snr, level, debounce)
              --> first crossing time, back-dated; NaN if never crossed
    |
    one row per shot: shot, isok_rm, whichn, ta_rm, tb_rm, ip_mean,
                      ft_nrmse, floor, rm2.0000, rm2.2500, ... rm30.0000
--> DataFrame.to_csv(out_csv)
```

The CSV is the MAST-U analogue of `fdp-regen-brm-plain-*.csv` and is where this pipeline currently ends. The next stage — feeding it, with a locked-mode counterpart, into the joint annotation of Section 3 — is not yet written.

Two independent entry points exist alongside the driver:

- `python saddle_analysis.py --shot-min 47000 --shot-max 47100` reproduces the original batch sweep (unchanged output format, plus the `.dropped.json` sidecar).
- `python check_equivalence.py --shot 47000` asserts on real data that `amplitude_with_coherence` still matches `RecognizedModes.amplitude()`, prints the array geometry and sample rate, and reports the coherence distribution against $1/N_c$. This is the first thing to run on Freya.

9.8 Base rate versus threshold on the existing data

Running the retrofitted trigger over the 78 collected shots (70.1 s of total dwell, $n = 1$, 8 ms debounce, crude $[0.05, 0.95]$ s window standing in for the flat-top) reproduces the DIII-D screening diagnostic of Section 2.4:

SNR level	events	fraction of shots	base hazard [s^{-1}]
2.0	61	0.782	0.870
3.0	54	0.692	0.771
4.0	49	0.628	0.699
5.0	47	0.603	0.671
7.0	41	0.526	0.585
10.0	19	0.244	0.271
15.0	5	0.064	0.071
20.0	0	0.000	0.000
30.0	0	0.000	0.000

There is a knee between SNR ≈ 7 and 15, which is the encouraging part: noise-floor normalisation alone already yields a usable ladder. The rates are roughly five times the DIII-D base rate, consistent with an ungated detector still counting noise excursions. The correct next measurement is to repeat this table with the coherence gate active and check whether the knee sharpens — exactly the "validate the base-rate curve first" advice of Section 8.

9.9 Calibration, in the MAST-U context

Section 7 distinguished the statistical hazard calibration from the hardware magnetics calibration. On MAST-U the hardware side is not yet done, and the code says so. In `RecognizedModes.amplitude`:

```
Ba = Bp/Fa/2/np.pi # TODO check
```

That comment is the whole issue. `Bp` $= \sqrt{\sum_f P(f,t)}$ comes from `matplotlib.mlab.specgram(mode='complex')`, which applies **no** window-power or sample-rate normalisation, and no coil area

or amplifier gain has been divided out. The DIII-D counterpart carries an explicit normalisation $\mathrm{ff} = 2/(p_w\, n_b\, n_{\mathrm{fft}})$, a factor T_s, and a final $\times 10^4$ that lands the result in Gauss. The MAST-U number lands at $\sim 2.3 \times 10^{-7}$ of nothing in particular.

There is also a formula difference worth recording. DIII-D weights each frequency bin by $1/\omega^2$ *inside* the sum,

$$A_n^{\mathrm{D3D}} = \sqrt{\sum_f \left(\tfrac{1}{\omega_f}\right)^2 \tfrac{1}{2}\left(X_{11} + X_{22}\right) \mathrm{ff}}, \tag{17}$$

whereas the MAST-U code divides the band total by the power-weighted centroid,

$$A_n^{\mathrm{MU}} = \frac{\sqrt{\sum_f P(f,t)}}{2\pi F_a}, \qquad F_a = \frac{\sum_f f\, P(f,t)}{\sum_f P(f,t)}. \tag{18}$$

These agree for a narrowband mode, where P is concentrated near F_a, and diverge for a broad or noisy spectrum — which, by Section 9.3, is the present regime.

Two consequences follow. Until the normalisation is settled, thresholds cannot be quoted as fields and cannot be compared against any published MAST-U or DIII-D number; and the statistical calibration of Section 7 is untouched by all of this but inherits its own MAST-U parameter, the EFIT++ slice interval Δ, which must be measured before any hazard is quoted in s^{-1}.

9.10 Stage-by-stage status

Stage	MAST-U status
Flat-top window	was absent; now written, needs validation on real I_p traces
RM spectral core	present, and arguably better than DIII-D: a global toroidal fit over the whole array rather than pairwise cross-spectra
Coherence gate	computed but discarded; now plumbed through and validated offline (Section 9.6)
Absolute calibration	missing entirely
Threshold ladder	was absent; now written
LM detector	no counterpart exists
Joint annotation	not started
EFIT (EPM/EPQ) features	not started — EPM is touched only for a max-I_p cut
Matched-pair merge	not started
Hazard model	not started

Three findings work in your favour. The OMAHA slow channels digitise at 200 kHz, *identical* to the DIII-D midplane probes, so the window-duration argument transfers without rescaling. `saddle_data.load_betan` tries `/epq/` and falls back to `/epm/`, which means **EPQ and EPM are the MAST-U counterparts of EFIT02 and EFIT01** — the matched-time-base study of Section 4 maps over intact, and the ± 5 ms matching logic can be reused as written. And the free pre-plasma baseline described above has no DIII-D equivalent.

9.11 Immediate next steps

1. Retrieve `omaha_coils.py` from Freya — the coil table is the array geometry and nothing can be validated without it — together with `saddle_geometry`, `mode_functions`, `pickup_coil_data` and `sxr_geometry`.

2. Validate `find_flattop` on a dozen hand-picked pulses in isolation, before any database-scale run. Mis-set flat-top constants are the most likely cause of an unexpectedly empty results table.

3. Settle the toroidal handedness (Section 9.6) by searching both signs on a shot with a known mode. Until this is known, a null result from `ntor = [1,2,3,4]` cannot be distinguished from a mode of the opposite sense.

4. Re-run the base-rate table with `--coherence-min` active and see whether the knee sharpens. This is the single measurement that will tell you whether the detector has become a detector.

5. Settle the amplitude normalisation, so the ladder can be expressed in Gauss.

6. Decide the locked-mode question. This is the real fork: without an M-matrix analogue the pathway model of Section 3.2 degenerates to `rotating-only` and `uneventful`, which is workable but discards the RM→LM ordering that motivates the two-detector design.

10 File map

File	Role
`fdp-run-study-plain.sh`	Top-level driver; threshold pinning, MD5 watermarking, stage sequencing, ML job submission
`fdp-regen-brm-blm-plain.sh`	Upstream detector regeneration (RM plain + classic, LM compensated + uncompensated)
`iptimestamp.py`	Flat-top window and flat-top quality metrics from I_p
`modespyec.py`	RM core: sliding cross-spectra, coherence, n-number, integrated Gauss amplitude, Schmitt/debounce trigger
`brm_events_modespyec.py`	`toksearch` pipeline and results tabulation for the RM ladder
`mpxspec.py`	Legacy even/odd-parity RM detector; also supplies `auxParseSingleTrace`
`blmEvents.py`	LM detector: M-matrix decomposition, coil compensation, dynamic rebaselining, LM ladder
`brmEventsCounter.py`, `blmEventsCounter.py`	Ladder → (threshold, counts, dwell, base hazard)
`show_brm_detectors.py`, `show_all_detectors.py`	Per-shot visual comparison of detector variants
`pathway_utils.py`	Pathway taxonomy; Markov dwell/jump/rate accumulator; profiled log-likelihood
`eventPathwayAnalysisPlain.py`	Join detectors; write joint annotation CSV; compute the threshold-plane surface
`checkPathwayPickle.py`	Render the threshold plane: likelihood, pathway fractions, dwell, rates, contours
`extractEqDescriptor.py`	EFIT descriptor harvest per tag
`extractEqProfiles.py`	`pres`/`qpsi` profile featurization
`processEqDescriptorPair.py`	±5 ms time matching of EFIT02 onto EFIT01; flattening; label placement
`tabular_limits.py`	Fixed and quantile screening limits

File	Role
scratch.py	Annotation index, pickle IO, key-respecting split primitives
xgb_hazard_common.py	Custom objective; base-rate anchoring; shot-respecting CV; outer-fold predictions; elimination paths; KL feature distances
xgbDescriptorModel.py	Main modeling program; plot_calibration; SHAP, partial dependence, per-shot traces
fdp-run-study-xgb-{ensembles,outerpreds, pooled,named}.sh	sbatch wrappers for the four experiment suites
testcalib.py	Standalone calibration, cumulative hazard, hazard-ratio and per-shot trace plots
test_beta_prior.py	Bayesian/bootstrap calibration histograms (PCLR model)
xgbShowElimination.py, xgbShowLinkage.py	Post-processing of elimination paths and feature linkage
MAST-U side (~/Dropbox/Python/Hazard)	
run_master.py	Driver: collect_signals per shot, detect_ladder / build_rm_detector, schmitt_first_crossing, CSV output
saddle_analysis.py	Original OMAHA routine, four defects repaired; saddle_analysis_one_shot, run_batch
saddle_extras.py	Additions: amplitude_with_coherence, noise_floor, analyze_shot, NTOR_BOTH_SIGNS
flattop.py	Flat-top window and quality metrics from I_p (the iptimestamp.py analogue)
check_equivalence.py	Asserts the reimplementation still matches upstream on real data
giopath.py	Locates Giovannozzi's Freya modules and puts them on sys.path; $GIOMAST_PATH overrides
egio/saddle_data.py	Giovannozzi's magnetics library: OMAHA loading, spectrogram, n-detection, amplitude